

Laporan Milestone 2 Tugas Besar

IF2224 Teori Bahasa Formal dan Automata

Arion Interpreter



K01 | Kelompok MLB

13524009 Mikhael Benrael Tampubolon

13524011 Muhammad Iqbal Raihan

13524025 Moh Hafizh Irham Perdana

13524099 Muhammad Akmal

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

Daftar Isi

Daftar Isi.....	2
BAB 1.....	3
1.1. Syntax Analysis dan Parser.....	3
1.2. Parse Tree.....	4
1.3. Algoritma Recursive Descent.....	5
BAB 2.....	7
2.1. Gambaran Umum Progam.....	7
2.2. Teknologi yang Digunakan.....	7
2.3. Struktur Program.....	8
2.4. Fungsi dan Kelas Utama.....	8
2.5. Alur Kerja Progam.....	10
BAB 3.....	12
3.1. Implementasi Parser Utama.....	12
3.2. Implementasi Parse Tree.....	13
3.3. Implementasi Modul Header dan Deklarasi Program.....	14
3.4. Implementasi Modul Subprogram.....	15
3.5. Implementasi Modul Statement.....	16
3.6. Implementasi Modul Ekspresi.....	17
3.7. Implementasi Diagnoser.....	17
BAB 4.....	19
4.1. Pengujian ke-1.....	19
4.2. Pengujian ke-2.....	19
4.3. Pengujian ke-3.....	20
4.4. Pengujian ke-4.....	20
4.5. Pengujian ke-5.....	20
BAB 5.....	21
Referensi.....	26

BAB 1

TEORI DASAR

1.1. Syntax Analysis dan Parser

Syntax analysis atau *parsing* adalah fase kedua dalam model atau proses kompilasi pada sebuah compiler. Jika diibaratkan, apabila *lexer* membaca karakter menjadi "kata", maka parser bertugas memeriksa apakah "kata-kata" tersebut membentuk sebuah "kalimat" yang valid sesuai tata bahasa. Proses ini biasanya bergantung pada sebuah himpunan aturan bernama Context-Free Grammar (CFG) untuk menentukan struktur sintaksis dari sebuah bahasa pemrograman.

Peran dan tanggung jawab utama dari sebuah parser di dalam compiler meliputi:

1. Pemeriksaan Sintaks (Syntax Checking): Memeriksa kebenaran sintaks/struktur dari aliran token (token stream) yang telah dihasilkan oleh *lexer*.
2. Membangun Representasi Struktur: Menangkap makna terstruktur dari sekumpulan token dan mengubahnya menjadi bentuk representasi seperti pohon atau hierarki, yang umumnya dikenal sebagai parse tree atau syntax tree. Pohon sintaks ini kemudian akan menjadi input bagi tahapan compiler selanjutnya (untuk menghasilkan intermediate representation).
3. Penanganan Kesalahan (Error Handling/Recovery): Parser bertugas mengidentifikasi jika kode sumber tidak memenuhi struktur yang benar (seperti titik koma yang salah tempat, kurang kurung kurawal, atau struktur operator yang salah). Parser dirancang sedemikian rupa untuk segera melaporkan error dengan jelas dan melakukan strategi pemulihan (error recovery, seperti panic-mode atau phrase-level recovery) agar dapat melanjutkan proses parsing dan menemukan sisa kesalahan lain di dalam kode.

Parser tidak dapat bekerja sebagai satu fungsi yang terpisah, melainkan memiliki keterkaitan yang sangat erat dan berkelanjutan dengan *Lexical Analyzer* atau *lexer*. *Lexer* dan *parser* saling berinteraksi bolak-balik. *Parser* bertindak sebagai pihak yang meminta elemen berikutnya kepada *lexer*. Sebagai balasan, *lexer* memindai kode sumber dan menyuplai token yang valid kepada *parser*. Token-token yang dihasilkan *lexer* inilah yang menjadi alfabet dasar (*terminal symbols*) yang akan digunakan *parser*.

Kemudian *parser* juga melengkapi keterbatasan *lexer*. *Lexer* bekerja menggunakan model ekspresi reguler (*regular expressions / finite automata*) yang bersifat terbatas. *Lexer* dapat membaca dan memvalidasi tipe token dengan baik, namun struktur ekspresi reguler tidak memiliki kemampuan untuk memvalidasi apakah pasangan token bersarang (seperti tanda kurung yang seimbang atau blok fungsi) tertulis dengan benar di dalam kode. Di sinilah *parser* mengambil alih peranan yang lebih luas untuk memeriksa batas struktural yang melampaui kapabilitas *lexer*

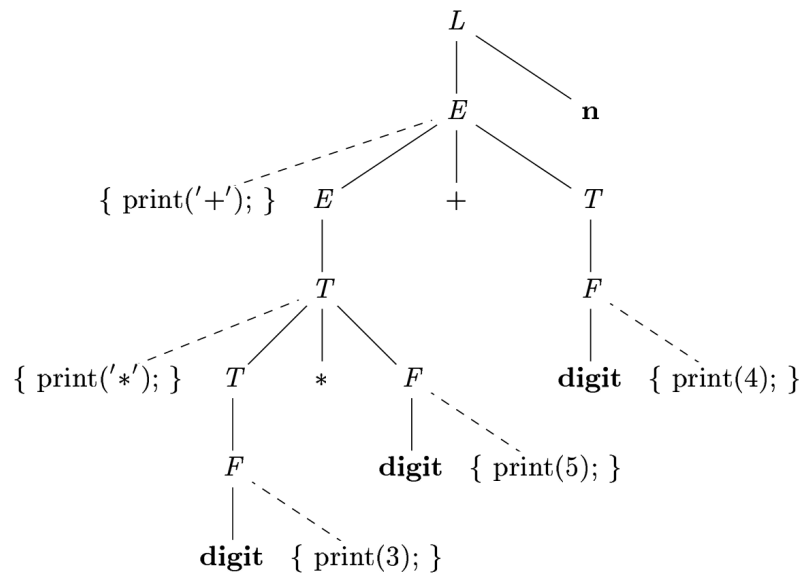
Lexer bertanggung jawab memvalidasi *token*, sedangkan *parser* menyusun *token* dari *lexer* tersebut terhadap tata bahasa (CFG). *Lexer* memverifikasi apakah token tersebut ada (misalnya memastikan suatu tipe karakter itu sah), tetapi tidak mengerti validitas urutannya secara komprehensif, sementara *parser* bertugas memahami operasi, urutan hierarkinya, dan validitas kombinasinya.

1.2. Parse Tree

Parse tree adalah representasi visual dari proses derivasi (penurunan) yang menunjukkan bagaimana sebuah *string* diturunkan dari simbol awal (*start symbol*) menggunakan aturan tata bahasa atau *grammar*. Dalam tahapan *Syntax Analysis*, *parse tree* merepresentasikan secara lengkap hierarki dan struktur sintaksis dari sebuah program sesuai dengan aturan *Context-Free Grammar* (CFG) yang digunakan. Pohon ini memuat semua simbol terminal dan non-terminal, sehingga sering juga disebut sebagai *concrete syntax tree* karena memperlihatkan secara pasti bagaimana sebuah teks diturunkan dari *grammar*.

Sebagai sebuah pohon (*tree*) berakar yang teratur (*rooted ordered tree*), *parse tree* memiliki struktur simpul (*node*) dengan properti spesifik berikut,

- Akar (*Root*): Simpul paling atas pada *parse tree* yang selalu direpresentasikan dan dilabeli oleh simbol awal (*start symbol*) dari *grammar*.
- Simpul Dalam (*Internal Nodes*): Merepresentasikan simbol-simbol non-terminal yang digunakan dalam aturan produksi. Setiap kali sebuah aturan produksi diterapkan, misalnya $A \rightarrow A_1 A_2 \dots A_n$, sebuah *internal node* akan dibuat untuk A , dan anak-anaknya (*children*) akan dibentuk dari bagian *body* (sisi kanan) aturan produksi tersebut yang dibaca dari kiri ke kanan
- Simpul Daun (*Leaf Nodes*): Merepresentasikan simbol terminal atau token yang merupakan unit dasar. Sebuah *leaf node* juga bisa berlabel *epsilon* (ϵ) apabila terdapat aturan produksi null ($A \rightarrow \epsilon$).



Secara umum, *Parse tree* sangat berguna untuk menunjukkan bagaimana token-token yang telah dihasilkan dari tahapan *lexical analysis* diorganisasikan untuk membentuk struktur program yang sah sesuai aturan *grammar*. Kemudian, *Parse tree* secara visual juga menggambarkan presedensi (hierarki prioritas) dan asosiativitas dari operator. Sub-pohon (*sub-tree*) yang posisinya paling dalam akan dievaluasi dan diselesaikan terlebih dahulu, sehingga operator pada node tersebut secara natural memiliki prioritas yang lebih tinggi dibandingkan operator di *node* induknya.

Peran lainnya adalah bahwa *Parse tree* dapat menjadi indikator ambiguitas. Jika sebuah *grammar* menghasilkan lebih dari satu *parse tree* yang berbeda untuk satu untaian input yang sama, maka *grammar* tersebut dikatakan ambigu (*ambiguous grammar*). Karena ambiguitas dapat menyulitkan kompilator dalam menerjemahkan kode, struktur *parse tree* menjadi instrumen penting untuk mendeteksi ambiguitas ini dan memastikan bahasa pemrograman dirancang agar terbebas dari tafsir ganda

1.3. Algoritma Recursive Descent

Algoritma Recursive Descent adalah algoritma top-down yang memproses input berdasarkan sebuah himpunan fungsi rekursif di mana setiap fungsi tersebut berkaitan dengan sebuah aturan atau grammar tertentu. Algoritma tersebut memproses input dari kiri ke kanan, kemudian membangun *parse tree* dengan mencocokkannya dengan aturan produksi suatu grammar. Algoritma ini cenderung mudah untuk diimplementasikan dan cocok untuk LL(1) grammar dimana setiap keputusan dapat dibuat hanya berdasarkan sebuah token tertentu.

Algoritma *recursive descent* diimplementasikan dengan membangun sekumpulan prosedur atau fungsi, di mana setiap fungsi mewakili satu simbol non-terminal secara spesifik dari aturan tata bahasa (*grammar*). Proses *parsing* dimulai dari inisialisasi, eksekusi program selalu dimulai dengan memanggil prosedur yang merepresentasikan simbol awal (*start symbol*) dari *grammar*. Kemudian dilakukan ekspansi produksi di dalam suatu fungsi non-terminal di mana algoritma akan memilih aturan produksi yang sesuai. Algoritma akan menelusuri simbol X_i satu per satu dari kiri ke kanan. Jika X adalah simbol non-terminal, maka prosedur untuk non-terminal tersebut akan dipanggil secara rekursif. Namun, jika X_i adalah simbol terminal, algoritma akan mencocokkannya dengan token input saat ini, lalu memajukan penunjuk input (*input pointer*) ke token berikutnya apabila cocok.

Namun dalam penerapannya, Algoritma Recursive Descent memiliki beberapa karakteristik dan batasan teoretis yang perlu diperhatikan. Bentuk umum dari *recursive descent* mungkin mengharuskan proses *backtracking* (mundur/kembali ke posisi input sebelumnya) apabila jalur aturan produksi yang dipilih ternyata gagal mencocokkan input. Versi yang lebih efisien dari metode ini adalah *predictive parser*, yang mampu memprediksi aturan produksi dengan tepat tanpa memerlukan *backtracking* sama sekali. *Predictive parser* biasanya dapat diterapkan pada tata bahasa berjenis LL(1).

Kelemahan lain dari algoritma *recursive descent* adalah ketidakmampuannya menangani tata bahasa yang mengandung rekursi kiri (*left-recursive grammar*). Jika terdapat aturan produksi berformat $A \rightarrow A\alpha$, algoritma akan terus-menerus memanggil fungsi $A()$ tanpa henti (*infinite loop*) karena tidak ada token terminal yang dikonsumsi. Oleh karena itu, *grammar* harus dimodifikasi terlebih dahulu untuk menghilangkan sifat rekursi kiri ini sebelum diimplementasikan ke dalam parser

BAB 2

PERANCANGAN PROGRAM

2.1. Gambaran Umum Program

Pada milestone 2, dilakukan implementasi parser sebagai kelanjutan dari lexer. Parser bertugas menerima deretan token yang telah dihasilkan oleh lexer pada milestone 1, kemudian memeriksa apakah susunan token tersebut sesuai dengan grammar bahasa Arion. Jika susunan token valid, parser membangun struktur hierarkis berupa parse tree yang merepresentasikan bentuk sintaksis program.

Parser dirancang menggunakan pendekatan recursive descent, yaitu setiap aturan grammar non-terminal direpresentasikan oleh satu fungsi parsing. Sebagai contoh, aturan program direpresentasikan oleh fungsi `parseProgram()`, aturan expression direpresentasikan oleh `parseExpression()`, dan aturan compound statement direpresentasikan oleh `parseCompoundStatement()`. Dengan rancangan ini, struktur kode parser mengikuti struktur grammar sehingga lebih mudah dibaca, diuji, dan dikembangkan.

2.2. Teknologi yang Digunakan

Parser diimplementasikan menggunakan bahasa C++17. Standar ini dipilih karena menyediakan fitur modern yang membantu pengelolaan struktur data kompleks, terutama dalam pembangunan parse tree. Beberapa fitur C++17 yang digunakan antara lain `std::vector`, `std::string`, `std::unique_ptr`, dan `std::optional`.

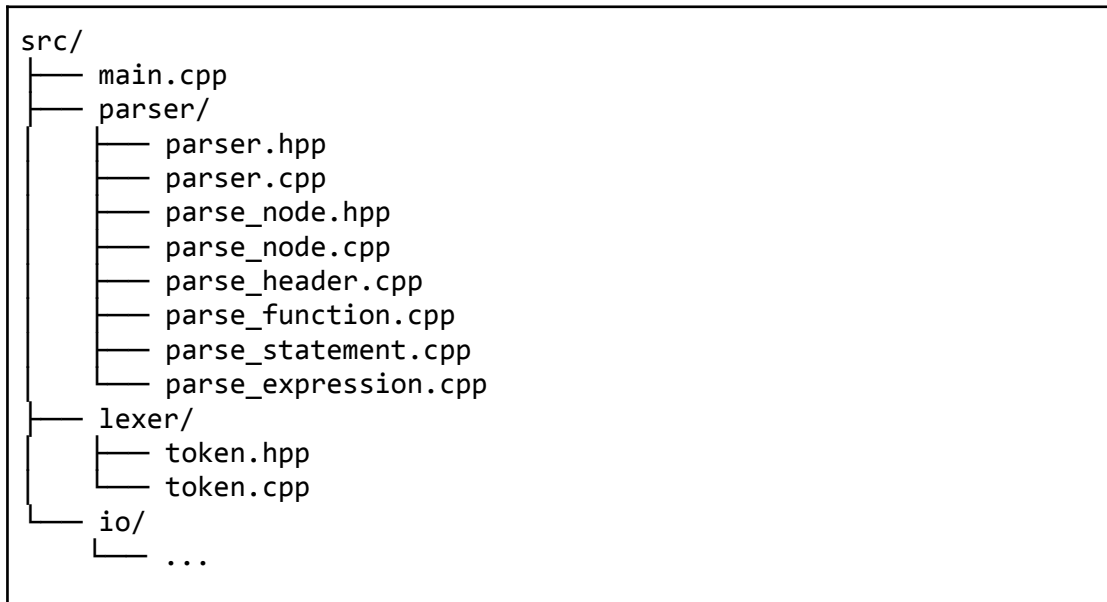
`std::vector` digunakan untuk menyimpan daftar token hasil lexer dan daftar child node pada parse tree. `std::unique_ptr` digunakan untuk mengelola kepemilikan node pada parse tree secara aman, sehingga setiap node memiliki kepemilikan eksklusif terhadap anak-anaknya. `std::optional` digunakan pada `ParseNode` karena tidak semua node menyimpan token. Node non-terminal seperti `Program` atau `Expression` tidak memiliki token langsung, sedangkan node terminal menyimpan token tertentu.

Sistem build menggunakan CMake. Pada rancangan proyek, parser dipisahkan ke dalam library tersendiri, yaitu `arion_parser`. Pemisahan ini membuat parser

menjadi modul independen yang dapat dikembangkan tanpa mencampur tanggung jawabnya dengan lexer ataupun komponen input/output.

2.3. Struktur Program

Struktur direktori untuk milestone 2 adalah sebagai berikut,



2.4. Fungsi dan Kelas Utama

Kelas utama pada milestone 2 adalah Parser. Kelas ini menyimpan nama file sumber, referensi terhadap daftar token, posisi token saat ini, dan root parse tree. Secara konseptual, **Parser** bekerja sebagai mesin pembaca token yang berjalan dari kiri ke kanan sambil membangun parse tree.

Atribut `tokens_` menyimpan referensi ke daftar token hasil lexer. Parser tidak menyalin token, melainkan hanya menggunakan referensi agar lebih efisien. Atribut `position_` menyimpan indeks token yang sedang dibaca. Setiap kali token berhasil dikonsumsi, nilai `position_` bertambah satu. Atribut `program_` merupakan akar parse tree dengan tipe node **Program**.


```

1  Parser::Parser(const std::string& filename,
2                  const std::vector<lexer::Token>& tokens)
3      : filename_(filename),
4        tokens_(tokens),
5        position_(0),
6        program_(NodeType::Program) {}

```

Fungsi `parse()` merupakan public API utama parser. Fungsi ini memulai proses parsing dengan memanggil `parseProgram()`. Karena simbol awal grammar adalah program, seluruh proses parsing selalu dimulai dari fungsi ini.

Fungsi `current()` mengembalikan token pada posisi saat ini. Fungsi ini digunakan ketika parser perlu menentukan aturan produksi mana yang harus dipilih berdasarkan token yang sedang dibaca. Fungsi `peek()` digunakan untuk melihat token berikutnya tanpa memajukan posisi parser. Fungsi ini berguna ketika parser perlu melakukan lookahead sederhana.

Fungsi `consume(expected)` adalah fungsi terpenting dalam proses pencocokan terminal. Fungsi ini memeriksa apakah token saat ini memiliki tipe yang sama dengan token yang diharapkan. Jika cocok, token tersebut dibungkus menjadi `ParseNode` bertipe `TokenNode`, lalu posisi parser dimajukan. Jika tidak cocok, fungsi ini melempar `UnexpectedTokenException`.

Kelas penting berikutnya adalah `ParseNode`. Kelas ini merepresentasikan satu simpul dalam parse tree. Setiap simpul memiliki `NodeType`, token opsional, dan daftar anak. Untuk simpul non-terminal, token bernilai kosong dan node memiliki child yang merepresentasikan hasil penurunan grammar. Untuk simpul terminal, node bertipe `TokenNode` dan menyimpan token yang dikonsumsi parser.

Adapun fungsi-fungsi utama untuk parsing adalah sebagai berikut,

Nama Fungsi	Keterangan
<code>parseProgram()</code>	Merepresentasikan aturan grammar paling atas. Fungsi ini memanggil parser untuk program header, declaration part, compound statement, dan diakhiri dengan token period. Setelah period dikonsumsi, parser memeriksa

	apakah masih ada token tersisa. Jika masih ada token setelah akhir program, input dianggap tidak valid.
<code>parseProgramHeader()</code>	Fungsi memvalidasi bagian awal program yang terdiri dari keyword program, identifier nama program, dan semicolon. Fungsi ini memastikan setiap program Arion memiliki header yang sesuai format.
<code>parseDeclarationPart()</code>	Fungsi menangani bagian deklarasi. Bagian ini mencakup deklarasi konstanta, tipe, variabel, serta subprogram. Rancangan ini mengikuti grammar Pascal-like, di mana deklarasi program ditempatkan sebelum compound statement utama.
<code>parseCompoundStatement()</code> , <code>parseStatementList()</code> , <code>parseStatement()</code>	Fungsi pada <code>parse_statement.cpp</code> dirancang untuk mengenali berbagai bentuk statement. <code>parseCompoundStatement()</code> menangani blok yang diawali <code>begin</code> dan diakhiri <code>end</code> . <code>parseStatementList()</code> menangani daftar statement yang dipisahkan oleh semicolon. <code>parseStatement()</code> menjadi fungsi pemilih yang menentukan jenis statement berdasarkan token saat ini.
<code>parseExpression()</code> , <code>parseSimpleExpression()</code> , <code>parseTerm()</code> , <code>parseFactor()</code>	Fungsi pada <code>parse_expression.cpp</code> dirancang berlapis untuk menangani prioritas operator. <code>parseExpression()</code> berada pada level tertinggi dan menangani operator relasional. <code>parseSimpleExpression()</code> menangani operator penjumlahan, pengurangan, dan <code>or</code> . <code>parseTerm()</code> menangani operator perkalian, pembagian, <code>div</code> , <code>mod</code> , dan <code>and</code> . <code>parseFactor()</code> menangani elemen dasar ekspresi seperti identifier, literal, ekspresi dalam kurung, dan unary operator.

2.5. Alur Kerja Progam

Alur kerja parser dimulai setelah lexer selesai menghasilkan daftar token. Objek Parser dibuat dengan menerima nama file sumber dan daftar token tersebut. Ketika fungsi `parse()` dipanggil, parser mulai membaca token dari indeks pertama.

Parser membaca token secara berurutan menggunakan `current()`. Untuk setiap aturan grammar, parser membuat node non-terminal yang sesuai, lalu

menambahkan child berdasarkan token atau hasil pemanggilan fungsi parsing lain. Jika aturan grammar membutuhkan token terminal tertentu, parser menggunakan `consume()`.

Sebagai contoh, ketika parser membaca program header, parser mengharapkan token `PROGRAMSY`, kemudian `IDENT`, lalu `SEMICOLON`. Jika ketiganya sesuai, node `ProgramHeader` terbentuk dengan tiga child terminal. Jika salah satu token tidak sesuai, parser langsung melaporkan kesalahan sintaks.

Pada bagian ekspresi, parser menggunakan pembagian fungsi berdasarkan prioritas operator. Dengan cara ini, struktur parse tree secara alami menunjukkan urutan evaluasi. Operator dengan prioritas lebih tinggi berada pada subtree yang lebih dalam, sedangkan operator dengan prioritas lebih rendah berada pada level yang lebih atas.

Setelah seluruh aturan berhasil diproses, parse tree tersimpan pada atribut `program_`. Parse tree ini dapat dicetak atau digunakan sebagai dasar untuk tahap berikutnya, seperti semantic analysis atau interpretasi program.

BAB 3

HASIL IMPLEMENTASI

3.1. Implementasi Parser Utama

Modul utama parser terdiri dari file parser.hpp dan parser.cpp. File parser.hpp berisi deklarasi kelas Parser, sedangkan parser.cpp berisi implementasi fungsi dasar yang digunakan oleh seluruh proses parsing.



```
1  Parser::Parser(const std::string& filename,  
2                  const std::vector<lexer::Token>& tokens)  
3      : filename_(filename),  
4        tokens_(tokens),  
5        position_(0),  
6        program_(NodeType::Program) {}  
7  
8  void Parser::parse() { parseProgram(); }
```

Konstruktor Parser menerima dua parameter, yaitu nama file sumber dan daftar token. Atribut filename_ digunakan untuk kebutuhan pelaporan error. Atribut tokens_ menyimpan referensi ke daftar token hasil lexer. Atribut position_ menunjukkan posisi token yang sedang dibaca, sedangkan program_ menjadi root dari parse tree. Fungsi parse() menjadi public API utama parser. Fungsi ini memulai proses parsing dengan memanggil parseProgram().

Selain itu, modul ini juga mengimplementasikan fungsi utilitas penting seperti is_done(), current(), peek(), dan consume(). Fungsi is_done() mengecek apakah seluruh token telah dikonsumsi. Fungsi current() mengambil token saat ini, sedangkan peek() melihat token berikutnya tanpa memajukan posisi parser.

```

1  ParsePtr Parser::consume(lexer::TokenType expected) {
2      if (position_ >= tokens_.size()) {
3          throw EOFTokenLookupException(filename_);
4      }
5
6      if (current().type != expected) {
7          throw UnexpectedTokenException(filename_, expected, current());
8      }
9
10     return std::make_unique<ParseNode>(NodeType::TokenNode,
11                                         tokens_.at(position_++));
12 }

```

Fungsi consume ini bertugas mencocokkan token saat ini dengan token yang diharapkan grammar. Jika sesuai, token dibungkus menjadi node terminal pada parse tree. Jika tidak sesuai, parser melempar exception. Dengan cara ini, semua fungsi parsing memiliki mekanisme validasi token yang konsisten.

3.2. Implementasi Parse Tree

Modul parse tree terdiri dari file parse_node.hpp dan parse_node.cpp. Modul ini bertanggung jawab menyediakan struktur data untuk menyimpan hasil parsing. Pada parse_node.hpp, terdapat enum NodeType yang mendefinisikan seluruh jenis node yang dapat muncul dalam parse tree. Node tersebut mencakup simbol non-terminal seperti Program, ProgramHeader, DeclarationPart, Statement, Expression, Term, dan Factor, serta TokenNode untuk merepresentasikan simbol terminal.

Atribut type_ menyimpan jenis node. Atribut token_ bersifat opsional karena hanya terminal node yang menyimpan token. Node non-terminal tidak menyimpan token secara langsung, melainkan memiliki child node. Atribut children_ menyimpan daftar anak dari node tersebut. Kemudian, penambahan child dilakukan melalui fungsi addChild().

```

1  class ParseNode {
2  public:
3      ParseNode(NodeType type) : type_(type) {}
4      ParseNode(NodeType type, lexer::Token token) : type_(type), token_(token) {}
5
6      NodeType type() const { return type_; }
7      const std::optional<lexer::Token>& token() const { return token_; }
8      const std::vector<std::unique_ptr<ParseNode>>& children() const {
9          return children_;
10     }
11
12     void addChild(std::unique_ptr<ParseNode> child) {
13         children_.push_back(std::move(child));
14     }

```

File `parse_node.cpp` mengimplementasikan fungsi pencetakan dan representasi string untuk setiap node. Operator `<<` digunakan untuk mengubah `NodeType` menjadi teks yang dapat dibaca. Fungsi `print()` kemudian mencetak parse tree secara hierarkis menggunakan indentasi berdasarkan kedalaman node. Implementasi ini berguna untuk debugging dan visualisasi hasil parsing.

3.3. Implementasi Modul Header dan Deklarasi Program

File `parse_header.cpp` berisi implementasi parsing untuk bagian awal program Arion. Bagian ini mencakup program header, declaration part, deklarasi konstanta, deklarasi tipe, deklarasi variabel, array type, range, enumerated type, dan record type.

```

1  void Parser::parseProgram() {
2      program_.addChild(parseProgramHeader());
3      program_.addChild(parseDeclarationPart());
4      program_.addChild(parseCompoundStatement());
5      program_.addChild(consume(TokenType::PERIOD));
6
7      if (!is_done())
8          throw std::runtime_error("TODO: EXCEPTION masih ada sisa after period");
9  }

```

3.4. Implementasi Modul Subprogram

```
1 ParsePtr Parser::parseSubprogramDeclaration() {
2     auto node = std::make_unique<ParseNode>(NodeType::SubprogramDeclaration);
3     if (current().type == TokenType::PROCEDURESY) {
4         node->addChild(parseProcedureDeclaration());
5     } else if (current().type == TokenType::FUNCTIONSY) {
6         node->addChild(parseFunctionDeclaration());
7     } else {
8         throw UnexpectedTokenException(filename_, TokenType::PROCEDURESY, current());
9     }
10    return node;
11 }
12
13 ParsePtr Parser::parseProcedureDeclaration() {
14     auto node = std::make_unique<ParseNode>(NodeType::ProcedureDeclaration);
15     node->addChild(consume(TokenType::PROCEDURESY));
16     node->addChild(consume(TokenType::IDENT));
17     if (current().type == TokenType::LPARENT) {
18         node->addChild(parseFormalParameterList());
19     }
20     node->addChild(consume(TokenType::SEMICOLON));
21     node->addChild(parseBlock());
22     node->addChild(consume(TokenType::SEMICOLON));
23     return node;
24 }
25
26 ParsePtr Parser::parseFunctionDeclaration() {
27     auto node = std::make_unique<ParseNode>(NodeType::FunctionDeclaration);
28     node->addChild(consume(TokenType::FUNCTIONSY));
29     node->addChild(consume(TokenType::IDENT));
30     if (current().type == TokenType::LPARENT) {
31         node->addChild(parseFormalParameterList());
32     }
33     node->addChild(consume(TokenType::COLON));
34     node->addChild(consume(TokenType::IDENT));
35     node->addChild(consume(TokenType::SEMICOLON));
36     node->addChild(parseBlock());
37     node->addChild(consume(TokenType::SEMICOLON));
38     return node;
39 }
```

File `parse_function.cpp` berisi fungsi parsing untuk procedure dan function. Modul ini disiapkan agar parser dapat mengenali deklarasi subprogram dalam program Arion.

Fungsi `parseSubprogramDeclaration()` menjadi fungsi pemilih antara procedure declaration dan function declaration. Jika token saat ini adalah `proceduresy`, parser akan memanggil `parseProcedureDeclaration()`. Jika token saat ini adalah `functionsy`, parser akan memanggil `parseFunctionDeclaration()`.

3.5. Implementasi Modul Statement

File `parse_statement.cpp` berisi fungsi-fungsi parsing untuk berbagai jenis statement dalam bahasa Arion. Modul ini menangani bagian program yang berada di dalam blok `begin ... end`.

```
1  ParsePtr Parser::parseStatement() {
2      auto node = std::make_unique<ParseNode>(NodeType::Statement);
3      switch (current().type) {
4          case TokenType::IFSY:
5              node->addChild(parseIfStatement());
6              break;
7          case TokenType::CASESY:
8              node->addChild(parseCaseStatement());
9              break;
10         case TokenType::WHILESY:
11             node->addChild(parseWhileStatement());
12             break;
13         case TokenType::REPEATSY:
14             node->addChild(parseRepeatStatement());
15             break;
16         case TokenType::FORSY:
17             node->addChild(parseForStatement());
18             break;
19         case TokenType::IDENT: {
20             // ident lparent begins a call; otherwise parse a variable and require
21             // becomes (:=) on the LHS.
22             if (!is_done() && peek(1).type == TokenType::LPARENT) {
23                 node->addChild(parseFunctionCall());
24             } else {
25                 ParsePtr var = parseVariable();
26                 if (current().type == TokenType::BECOMES) {
27                     auto asn =
28                         std::make_unique<ParseNode>(NodeType::AssignmentStatement);
29                     asn->addChild(std::move(var));
30                     asn->addChild(consume(TokenType::BECOMES));
31                     asn->addChild(parseExpression());
32                     node->addChild(std::move(asn));
33                 } else {
34                     throw UnexpectedTokenException(filename_, TokenType::BECOMES,
35                                                         current());
36                 }
37             }
38             break;
39         }
40         case TokenType::SEMICOLON:
41         case TokenType::ENDSY:
42         case TokenType::UNTILSY:
43         case TokenType::ELSESY:
44             break;
45         default:
46             throw UnexpectedTokenException(filename_, TokenType::IDENT, current());
47     }
48     return node;
49 }
```


3.6. Implementasi Modul Ekspresi

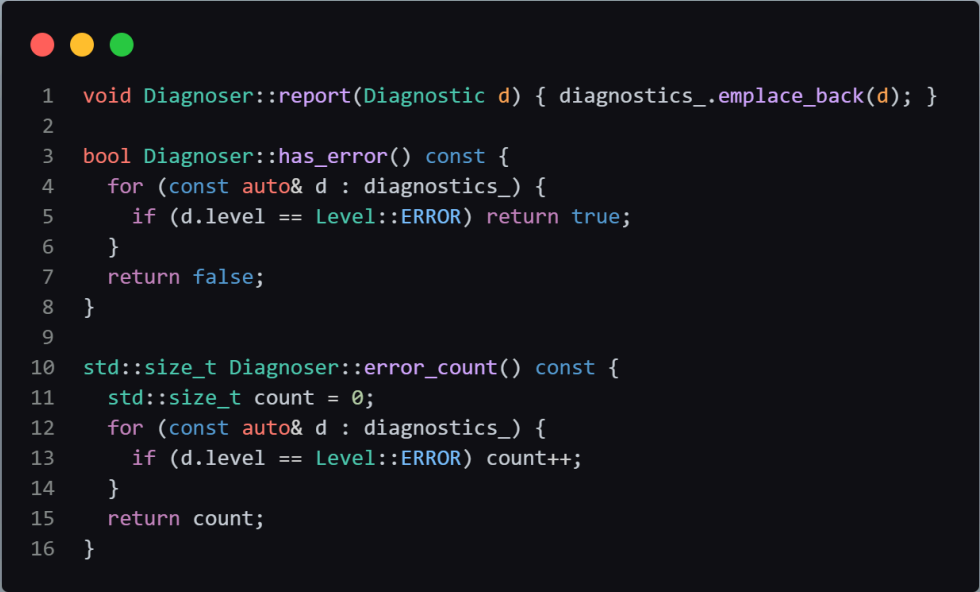
File `parse_expression.cpp` berisi implementasi parsing ekspresi. Ekspresi dipisahkan menjadi beberapa tingkat fungsi agar parser dapat merepresentasikan prioritas operator dengan benar.

```
1 ParsePtr Parser::parseExpression() {
2     auto node = std::make_unique<ParseNode>(NodeType::Expression);
3     node->addChild(parseSimpleExpression());
4
5     auto t = current().type;
6     if (t == lexer::TokenType::EQL || t == lexer::TokenType::NEQ ||
7         t == lexer::TokenType::GTR || t == lexer::TokenType::GEQ ||
8         t == lexer::TokenType::LSS || t == lexer::TokenType::LEQ) {
9         node->addChild(parseRelationalOperator());
10        node->addChild(parseSimpleExpression());
11    }
12    return node;
13 }
14
15 ParsePtr Parser::parseSimpleExpression() {
16     auto node = std::make_unique<ParseNode>(NodeType::SimpleExpression);
17
18     auto t = current().type;
19     if (t == lexer::TokenType::PLUS || t == lexer::TokenType::MINUS) {
20         node->addChild(consume(t));
21     }
22
23     node->addChild(parseTerm());
24
25     t = current().type;
26     while (t == lexer::TokenType::PLUS || t == lexer::TokenType::MINUS || t == lexer::TokenType::ORSY) {
27         node->addChild(parseAdditiveOperator());
28         node->addChild(parseTerm());
29         t = current().type;
30     }
31     return node;
32 }
```

Fungsi `parseExpression()` berada pada abstraksi paling atas dan menangani ekspresi yang dapat memiliki operator relasional. Fungsi `parseSimpleExpression()` menangani ekspresi sederhana yang dapat diawali tanda plus atau minus, kemudian diikuti satu atau lebih term yang dipisahkan operator aditif.

3.7. Implementasi Diagnoser

`diagnoser.hpp` dan `diagnoser.cpp` mengimplementasikan kelas `Diagnoser`. Kelas ini menyimpan daftar diagnostic yang dilaporkan selama proses lexer dan parser. Fungsi `report()` digunakan untuk menambahkan diagnostic baru. Fungsi `has_error()` digunakan untuk memeriksa apakah terdapat error. Fungsi `error_count()` menghitung jumlah error.



```
1 void Diagnoser::report(Diagnostic d) { diagnostics_.emplace_back(d); }
2
3 bool Diagnoser::has_error() const {
4     for (const auto& d : diagnostics_) {
5         if (d.level == Level::ERROR) return true;
6     }
7     return false;
8 }
9
10 std::size_t Diagnoser::error_count() const {
11     std::size_t count = 0;
12     for (const auto& d : diagnostics_) {
13         if (d.level == Level::ERROR) count++;
14     }
15     return count;
16 }
```

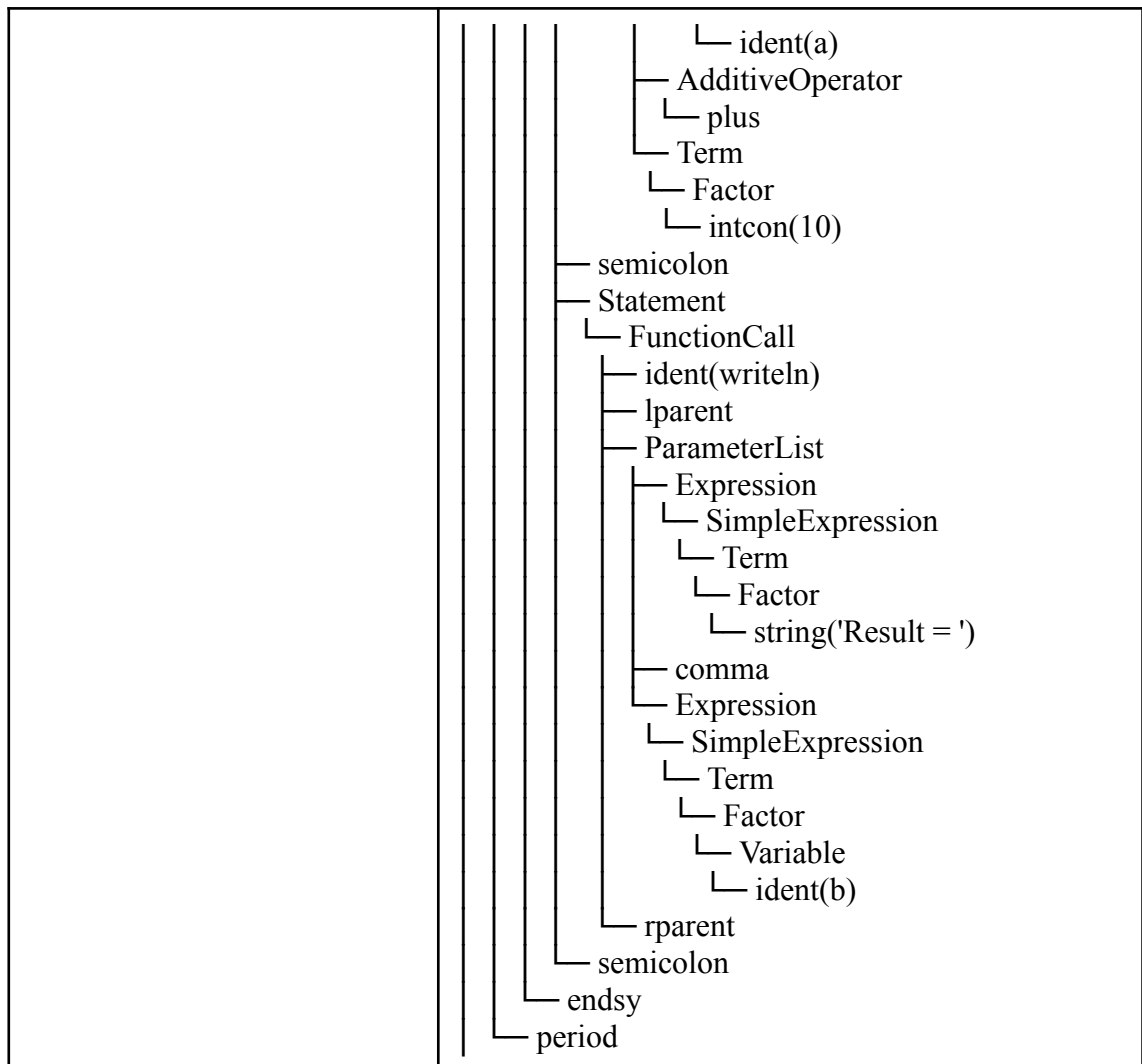
Dengan modul diagnoser, parser tidak perlu menghentikan program menggunakan exception setiap kali menemukan error sintaks. Parser cukup melaporkan error, melakukan recovery sederhana, dan melanjutkan proses jika memungkinkan.

BAB 4

PENGUJIAN

4.1. Pengujian ke-1

Input	Output
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>	<pre> ├─ Program │ └─ ProgramHeader │ ├── programsy │ ├── ident(Hello) │ └─ semicolon │ └─ DeclarationPart │ └─ VarDeclaration │ ├── varsy │ ├── IdentifierList │ │ ├── ident(a) │ │ ├── comma │ │ └─ ident(b) │ ├── colon │ ├── Type │ │ └─ ident(integer) │ └─ semicolon │ └─ CompoundStatement │ ├── beginsy │ └─ StatementList │ ├── Statement │ │ └─ AssignmentStatement │ │ ├── Variable │ │ │ └─ ident(a) │ │ ├── becomes │ │ └─ Expression │ │ └─ SimpleExpression │ │ └─ Term │ │ └─ Factor │ │ └─ intcon(5) │ ├── semicolon │ └─ Statement │ └─ AssignmentStatement │ ├── Variable │ │ └─ ident(b) │ ├── becomes │ └─ Expression │ └─ SimpleExpression │ └─ Term │ ├── Factor │ └─ Variable</pre>



4.2. Pengujian ke-2

Input	Output

4.3. Pengujian ke-3

Input	Output

4.4. Pengujian ke-4

4.5. Pengujian ke-5

BAB 5

ATURAN GRAMMAR

Berikut merupakan aturan produksi (*grammar*) yang digunakan dalam bahasa Arion.

No	Nama Variabel CFG	Deskripsi	Aturan Produksi	Contoh
1	<program>	<i>Start variable</i> yang merepresentasikan keseluruhan program Arion	program → program-header + declaration-part + compound-statement + period	Seluruh struktur program dari awal hingga akhir
2	<program-header>	Bagian kepala program yang berisi nama program	programsy + ident + semicolon	program Hello;
3	<declaration-part>	Bagian deklarasi yang dapat berisi const, type, var, procedure, function	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*	Semua deklarasi sebelum begin
4	<const-declaration>	Deklarasi konstanta	constsy + (ident + eql + constant + semicolon)+	const INT == 100; REAL == 10.0; CHAR == 'a'; STRING == 'abcd';
5	<constant>	Konstanta dapat berupa ident , intcon , realcon , charcon , atau string	charcon string [(plus minus)? + (ident intcon realcon)]	'a' 'abcd' 10 +10.0 -a
6	<type-declaration>	Deklarasi tipe data baru	typesy + (ident + eql + type + semicolon)+	type Range == 1..10; type Month == (Jan, Feb, Mar, Apr);
7	<var-declaration>	Deklarasi variabel	varsy + (identifier-list + colon + type + semicolon)+	var a, b: integer;
8	<identifier-list>	Daftar identifier yang dipisahkan koma	ident (comma + ident)*	a, b, c

9	<type>	Tipe data yang terdefinisi atau didefinisikan pengguna. Dapat berupa ident , array-type, range, atau enumerated	ident array-type range enumerated record-type	integer , array[1..10] of integer
10	<array-type>	Definisi tipe array	arraysy + lbrack + (range ident) + rbrack + ofsy + type	array[1..10] of integer array[char] of integer
11	<range>	Rentang nilai untuk array atau subrange	constant + period + period + constant	1..10 , 'a'..'z'
12	<enumerated>	Nilai tertentu yang ada di dalam list	lparent + ident + (comma + ident)* + rparent	(Jan, Feb, Mar, Apr)
13	<record-type>	Tipe data template, sama dengan <i>class</i> dalam OOP	recordsy + field-list + endsy	type record-name = record field-1: field-type1; field-2: field-type2; ... field-n: field-typen; end
14	<field-list>	Penjabaran atribut/field dari record	field-part + (semicolon + field-part)*	..field-1..; ..field-2..
15	<field-part>	Singular atribut/field dari record	identifier-list + colon + type	field-1: field-type1;
16	<subprogram-declaration>	Deklarasi prosedur atau fungsi	procedure-declaration function-declaration	procedure print(x: integer);
17	<procedure-declaration>	Deklarasi prosedur	proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon	Prosedur dengan parameter opsional

18	<function-declaration>	Deklarasi fungsi	function sy + ident + (formal-parameter-list)? + colon + ident + semicolon + block + semicolon	Fungsi yang mengembalikan nilai
19	<block>	Bagian kode yang mengandung deklarasi dan statement	declaration-part + compound-statement	var ... begin ... end
20	<formal-parameter-list>	Daftar parameter formal	lparent + parameter-group + (semicolon + parameter-group)* + rparent	(x, y: integer; z: real)
21	<parameter-group>	Daftar parameter disertai dengan tipe data parameter	identifier-list + colon + (ident array-type)	x, y: integer
22	<compound-statement>	Blok statement yang diawali begin dan diakhiri end	begin sy + statement-list + endsy	begin ... end
23	<statement-list>	Daftar statement yang dipisahkan semicolon	statement (semicolon + statement)*	Urutan statement dalam blok
24	<statement>	Deskripsi aksi yang harus dilakukan program komputer	(assignment-statement if-statement case-statement while-statement repeat-statement for-statement procedure/function-call)?	x := 5; if x > 0 then y := 1 else y := 0 while x < 10 do x := x + 1
25	<variable>	Variable dapat berbentuk ident, elemen array, atau field record	ident + (component-variable)*	someVar someArray[1] someField
26	<component-variable>	Variable dalam bentuk element array atau field record	(lbrack + index-list + rbrack) (period + ident)	someArray[1] someRecord.someField someArray[1][2]
27	<index-list>	Penunjuk index dalam array	(intcon charcon ident) + (comma + index-list)*	[1, 2, 3] ['a', 'b'] [true, false]

28	<assignment-statement>	Statement penugasan nilai ke variabel	variable + becomes + expression	x := 5, y := x + 10
29	<if-statement>	Statement kondisional	ifsy + expression + thensy + statement + (elsy + statement)?	if x > 0 then y := 1 else y := 0
30	<case-statement>	Statement kondisional dengan banyak kondisional	casesy + expression + ofsy + case-block + endsy	case i of 0: x := 0; 1 : x := 1; end
31	<case-block>	Deklarasi case dan statement yang harus dilakukan	constant + (comma + constant)* + colon + statement + (semicolon + case-block?)*	0: x := 0; 1, 2: x := 1
32	<while-statement>	Perulangan dengan kondisi di awal	whilesy + expression + dosy + statement	while x < 10 do x := x + 1
33	<repeat-statement>	Perulangan dengan kondisi di akhir	repeatsy + statement-list + untilsy + expression	repeat x := x - 1 until x == 0
34	<for-statement>	Perulangan dengan counter	forsy + ident + becomes + expression + (tosy downtosy) + expression + dosy + statement	for i := 1 to 10 do ...
35	<procedure/function-call>	Pemanggilan prosedur atau fungsi	ident + (lparent + parameter-list? + rparent)?	writeln('Hello') , print(x, y)
36	<parameter-list>	Daftar parameter aktual saat pemanggilan	expression (comma + expression)*	'Result = ', b, 100
37	<expression>	Ekspresi yang menghasilkan nilai	simple-expression (relational-operator + simple-expression)?	x + y, a > b, 5 * (x + 1)
38	<simple-expression>	Ekspresi tanpa operator relasional	(plus minus)? term (additive-operator + term)*	x + y - z, 5 * 2
39	<term>	Bagian ekspresi dengan prioritas lebih tinggi	factor (multiplicative-operator + factor)*	x * y, a div b

40	<factor>	Unit terkecil dalam ekspresi	ident intcon realcon charcon string (lparent + expression + rparent) (notsy + factor) procedure/function-call variable	x, 5, 'a', (x+y), not a
----	-----------------------	------------------------------	---	-------------------------

Referensi

1. Spesifikasi Milestone 1 - Tubes IF2224 TBFO
2. Spesifikasi Milestone 2 - Tubes IF2224 TBFO
3. <https://pdfs.semanticscholar.org/e142/91a9afec31014a9d489e3ad35a7a612088df.pdf>
4. https://www.tutorialspoint.com/compiler_design/pdf/compiler_design_syntax_analysis.pdf
5. <https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>